Aly Muhammad Kazani 24K-0512

OS Lab Task 6

1)

```c
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#define FIFO_NAME "/tmp/myfifo"

int main() {
  int pid;
  int fd;
  char message[10];

  // Step 1: Create FIFO
  mkfifo(path: FIFO_NAME, mode: 0666);

  // Step 2: Fork process
  pid = fork();
  if (pid < 0) {
    printf(format: "Fork failed!\n");
    return 1;
  } else if (pid == 0) {

    // Child process → Reader
    fd = open(file: FIFO_NAME, oflag: O_RDONLY);
    read(fd, buf: message, nbytes: sizeof(message));

    printf(format: "Child received: %s\n", message);
    close(fd);
  } else {

    // Parent process → Writer
    printf(format: "Enter a message to send: ");
    fgets(s: message, n: sizeof(message), stream: stdin);
    fd = open(file: FIFO_NAME, oflag: O_WRONLY);
    write(fd, buf: message, n: strlen(s: message) + 1); // +1 for null terminator

    close(fd);
    printf(format: "Parent sent the message!\n");
  }

  return 0;
}
```

```
OS-Lab-Tasks/lab6 main ? ❯ gcc task1.c -o t1

OS-Lab-Tasks/lab6 main ? ❯ ./t1
Enter a message to send: hello
Parent sent the message!
Child received: hello


OS-Lab-Tasks/lab6 main ? ❯ # without changes ^

OS-Lab-Tasks/lab6 main ? ❯ gcc task1.c -o t1

OS-Lab-Tasks/lab6 main ? ❯ ./t1
Enter a message to send: buffer 10 but string larger
Parent sent the message!
Child received: buffer 10

OS-Lab-Tasks/lab6 main ? ❯ # with 10 buffer size

OS-Lab-Tasks/lab6 main ? ❯
```

```c
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#define FIFO_NAME "/tmp/myfifo"

int main() {
  int pid;
  int fd;
  char message[100];

  // Step 1: Create FIFO
  mkfifo(path: FIFO_NAME, mode: 0666);

  // Step 2: Fork process
  pid = fork();
  if (pid < 0) {
    printf(format: "Fork failed!\n");
    return 1;
  } else if (pid == 0) {

    // Child process → Reader
    fd = open(file: FIFO_NAME, oflag: O_RDONLY);
    read(fd, buf: message, nbytes: sizeof(message));

    printf(format: "Child received: %s\n", message);
    close(fd);
  } else {

    // Parent process → Writer
    printf(format: "Enter a message to send: ");
    fgets(s: message, n: sizeof(message), stream: stdin);
    int *msg = (void *)message; // convert to int* type
    fd = open(file: FIFO_NAME, oflag: O_WRONLY);
    write(fd, buf: msg, n: sizeof(msg)); // +1 for null terminator

    close(fd);
    printf(format: "Parent sent the message!\n");
  }

  return 0;
}
```

```
OS-Lab-Tasks/lab6 main ? ❯ gcc task1.c -o t1

OS-Lab-Tasks/lab6 main ? ❯ ./t1
Enter a message to send: hello
Parent sent the message!
Child received: hello


OS-Lab-Tasks/lab6 main ? ❯ # without changes ^

OS-Lab-Tasks/lab6 main ? ❯ gcc task1.c -o t1

OS-Lab-Tasks/lab6 main ? ❯ gcc task1.c -o t1

OS-Lab-Tasks/lab6 main ? ❯ ./t1
Enter a message to send: writing numbers
Parent sent the message!
Child received: writing

OS-Lab-Tasks/lab6 main ? ❯ gcc task1.c -o t1

OS-Lab-Tasks/lab6 main ? ❯ ./t1
Enter a message to send: writing numbers
Parent sent the message!
Child received: writing

OS-Lab-Tasks/lab6 main ? ❯ # conclusion, 'write' writes raw bytes and doesnt care abt the type, rest is m
issing beuz of the space in the middle of the words.

OS-Lab-Tasks/lab6 main ? ❯
```

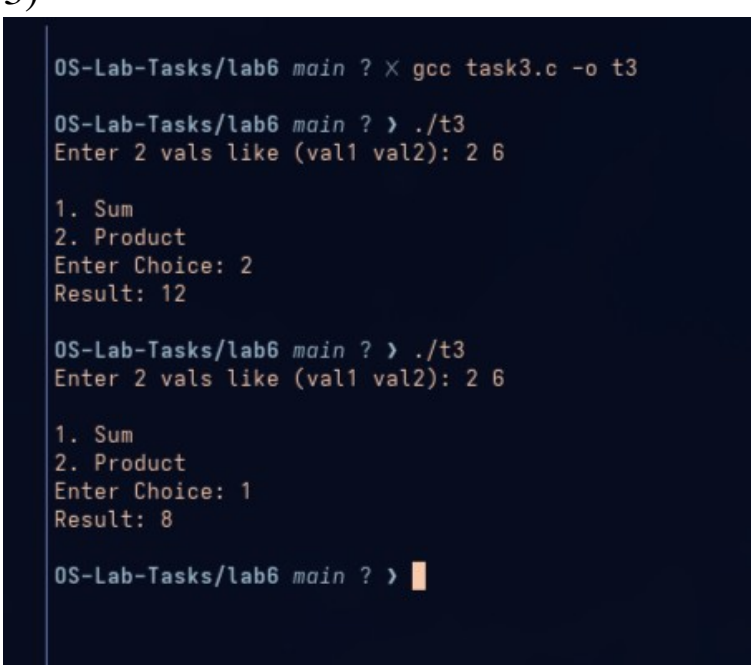
2)

```
OS-Lab-Tasks/lab6 main ? ✗ gcc task2.c -o t2

OS-Lab-Tasks/lab6 main ? ❯ ./t2
Enter a message to send: hello
Parent sent: hello
Printed in reverse: olleh

OS-Lab-Tasks/lab6 main ? ❯
```

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <unistd.h>

#define FIFO_NAME "/tmp/myfifo"

int main() {
  pid_t pid;
  int fd;
  char message[50];

  mkfifo(FIFO_NAME, 0666);

  pid = fork();

  if (pid < 0) {
    perror("fork");
    exit(EXIT_FAILURE);
  }

  if (pid > 0) { // parent
    printf("Enter a message to send: ");
    fgets(message, sizeof(message), stdin);

    fd = open(FIFO_NAME, O_WRONLY);

    write(fd, message, strlen(message) + 1);
    printf("Parent sent: %s", message);

    close(fd);
```

```
    wait(NULL);
    unlink(FIFO_NAME);
  }

  if (pid == 0) { // child
    fd = open(FIFO_NAME, O_RDONLY);

    read(fd, message, sizeof(message));

    close(fd);

    int len = strlen(message);
    if (len > 0 && message[len - 1] == '\n')
      len--;

    printf("Printed in reverse: ");
    for (int i = len - 1; i >= 0; i--) {
      putchar(message[i]);
    }
    printf("\n");

    exit(EXIT_SUCCESS);
  }

  return EXIT_SUCCESS;
}
```

3)



```
#include <fcntl.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <unistd.h>

#define FIFO_NAME "/tmp/myfifo"

int main(void) {

  int fd, val1, val2;
  char buffer[100];
  pid_t pid;

  mkfifo(FIFO_NAME, 0666);

  pid = fork();
  if (pid < 0) {
    perror("fork");
    exit(EXIT_FAILURE);
  }

  if (pid > 0) { // parent
    printf("Enter 2 vals like (val1 val2): ");
    if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
      fd = open(FIFO_NAME, O_WRONLY);

      write(fd, buffer, sizeof(buffer));

      close(fd);
    }

    wait(NULL);
    unlink(FIFO_NAME);
  }

  if (pid == 0) { // child
    int choice;
    fd = open(FIFO_NAME, O_RDONLY);
```

```
    read(fd, buffer, sizeof(buffer));

    close(fd);

    sscanf(buffer, "%d %d", &val1, &val2);

  again:
    printf("\n1. Sum\n2. Product\n");
    printf("Enter Choice: ");
    if (scanf("%d", &choice) != 1) {
      while (getchar() != '\n')
        ;
      goto again;
    }

    switch (choice) {
    case 1:
      printf("Result: %d\n", (val1 + val2));
      break;
    case 2:
      printf("Result: %d\n", (val2 * val1));
      break;
    default:
      printf("Invlaid Choice! Try Again.\n");
      goto again;
    }
  }

  return EXIT_SUCCESS;
}
```